

# legacy-reverse 利用マニュアル

## 目次

|                                            |    |
|--------------------------------------------|----|
| 0. 用意するもの（前提条件） .....                      | 4  |
| 1. skill の配置 .....                         | 4  |
| 2. hook の登録（必須） .....                      | 5  |
| 3. MCP サーバの登録（推奨） .....                    | 5  |
| 4. ツール類 .....                              | 5  |
| 5. 開始 .....                                | 6  |
| 関数リストはあとから調整できる（追加・対象外） .....              | 7  |
| ①②の承認とは何をすることか .....                       | 8  |
| 実行はチャットか CLI から（1 関数ずつ／まとめて） .....         | 10 |
| バッチ実行状況ページ — 進捗のライブ表示（見るだけ） .....          | 11 |
| ①はまとめて進めて、まとめてレビューできる .....                | 12 |
| ⑤で fail したときの裁定 .....                      | 14 |
| なぜ辞書が先なのか .....                            | 16 |
| 変数辞書の作られ方 .....                            | 16 |
| 承認のしかた .....                               | 16 |
| あとから変数が増えたとき .....                         | 17 |
| 例外ポリシー — 0 割をどう扱うか .....                   | 17 |
| exception-queue への答え方 .....                | 17 |
| WBS は自動でダッシュボードになる .....                   | 20 |
| 中断しても 1 からやり直しにならない .....                  | 21 |
| WBS サイト .....                              | 22 |
| レビュアーへの配布（実行するだけの EXE） .....               | 23 |
| あなたが書くファイル（AI は提案まで） .....                 | 23 |
| AI への指示をプロジェクトごとに調整する（docs/prompts/） ..... | 24 |

レガシーコード（Fortran / C / C++ など）を Python へ仕様ベースで移植するための、Claude Code skill 群です。関数を 1 つずつ、次の流れで進めます。

**対応言語について:** ⑥の関数列举を機械抽出できるのは現在 **Fortran と C/C++** です（両方が混在していても、同じ関数リストにマージされます）。それ以外の言語（COBOL・C# など）でも①以降の進め方は同じですが、⑥の関数列举は AI と人で行うことになります。

⑥リポジトリ解析 → ①関数仕様書 → ②テスト仕様書 → ③テストコード → ④実装 → ⑤テスト  
→ ⑥完了検証 → ⑦分析・改善

設計の柱は 5 つあります。

1. **クリーンルーム分業** — ②③はレガシー原文を見ない。③は「②+規約」だけ、④は「①+規約」だけを入力に作られる。独立に作ったテストと実装を⑤で突き合わせることで、仕様書の穴が機械的に炙り出される。
2. **ハッシュ連鎖** — ①→②→③の各成果物は上流のハッシュを記録している。上流が改訂されると下流が自動的に「要再生成 (stale△)」になり、伝搬漏れが起きない。
3. **人の承認ゲート** — 仕様の確定・テスト仕様の承認・裁定は必ず人が行う。AI は「仮説+Yes/No の問い」の形で判断材料を出し、勝手に確定しない。
4. **機械が正、AI は意味づけ** — 関数の列举（⑥）は静的解析プログラムが行い、AI の成果物（①②）は人に届く前に機械レビュー（根拠引用の实在検証・省略検知・トレーサビリティ突合）を通る。「もっともらしいが根拠のない記述」はあなたの目に触れる前に機械が落とす（詳細は「品質ゲート」の章）。
5. **挙動保存の改善（⑦）** — 移植完了後の高速化・リファクタリングは、③のテスト資産を安全網に「テスト全 pass 維持」を絶対条件として行う。

**あなた（人）の仕事は「トリガを引く」「質問に答える」「承認する」「裁定する」の 4 つです。** コードや文書を直接書く必要は基本的にありません（書いてもよい場所は後述）。

**前提: レガシーを動かせる環境は無くてもよい**（あるならなお良い、という位置づけです）。このパイプラインは「レガシーを実行して出力を突き合わせる」方式ではありません。正しさは **(1) 人が仕様書をレビューして確定すること**と、**(2) その仕様書だけから独立に作ったテストと実装が⑤で一致すること**で担保します。②で「期待値が決められない」ケースが出てあなたに質問が来るのは、この前提のためです（レガシーを走らせて実測できるなら、そう答えてもらって構いません）。

裏を返すと、**最終的な品質の上限は①仕様書レビューの質**です。⑤の pass は「あなたが承認した仕様どおりに実装されている」ことを保証しますが、「仕様があなたの承認どおりレガシーと同じ」かどうかは、あなたのレビューが決めます。

初めての人は [slides/index.html](https://slides/index.html)（スライド版・ブラウザで開くだけ）で ⑥→⑦を順に体験しながら覚えるのが早道です。作業中に手元へ置くコマンド一覧は [QUICKREF.md](#)（1 枚）。本書は背景と操作の意味を説明する「ドキュメント版」です。

# pricing-batch リバースエンジニアリング WBS

公開  
2026年7月25日

目次

[進捗サマリ](#)

[Open ISSUES（要人間判断）](#)

[関数一覧（推奨着手順）](#)

[⑦ 改善イタレーション](#)

## 進捗サマリ

| ①spec | ②test-spec | ③test-code | ④impl | ⑤test |
|-------|------------|------------|-------|-------|
| 3/3   | 3/3        | 3/3        | 3/3   | 3/3   |

## Open ISSUES（要人間判断）

なし 🚩

## 関数一覧（推奨着手順）

| # | 関数                         | 依存             | ①spec | ②test-spec | ③test-code | ④impl | ⑤test |
|---|----------------------------|----------------|-------|------------|------------|-------|-------|
| 1 | <a href="#">get_rate</a>   | なし             | ✅     | ✅          | ✅          | ✅     | ✅     |
| 2 | <a href="#">round_val</a>  | なし             | ✅     | ✅          | ✅          | ✅     | ✅     |
| 3 | <a href="#">calc_price</a> | F-0001, F-0002 | ✅     | ✅          | ✅          | ✅     | ✅     |

## ⑦ 改善イタレーション

| ID                      | 分類       | 目的                             | 対象     | 状態       | 達成 |
|-------------------------|----------|--------------------------------|--------|----------|----|
| <a href="#">REF-001</a> | refactor | get_rate のフォールバック読みを分離し複雑度を下げる | F-0001 | verified | ✅  |

図 1: WBS（進捗のホーム画面）。進捗サマリ・あなたへの質問（Open ISSUES）・関数一覧・⑦改善イタレーションが 1 画面に集約され、各✅から成果物へ移動できる

## セットアップ

### 0. 用意するもの（前提条件）

Table 1

| 必要なもの                                    | 確認方法                          | 備考                                                                  |
|------------------------------------------|-------------------------------|---------------------------------------------------------------------|
| <b>Claude Code</b> （CLI が PATH に通っていること） | <code>claude --version</code> | チャットからだけでなく <b>コマンドとして起動</b> できる必要があります。ブラウザ実行・無人バッチはこの CLI を裏で呼びます |
| <b>Python 3.10 以上</b>                    | <code>python --version</code> | スクリプト群の実行に使います                                                      |
| <b>移植先プロジェクトが git 管理下</b>                | <code>git status</code>       | 成果物の履歴管理と、事故ったときの復旧（ <code>git checkout --</code> ）に使います            |
| skills リポジトリ                             | —                             | この文書が入っているリポジトリ。 <b>セットアップ後も消さないでください</b> （MCP サーバがこの中を参照します）       |

この文書に出てくる書き方:

Table 2

| 表記        | 意味                                                                                      |
|-----------|-----------------------------------------------------------------------------------------|
| <project> | 移植先のプロジェクトのルート。レガシーコードと新実装を置く場所                                                         |
| <skills>  | skills リポジトリを clone した場所（例: <code>C:\work_space\skills</code> ）                         |
| <LR>      | <b>セットアップ後の skill のルート</b> = <code>&lt;project&gt;/.claude/skills/legacy-reverse</code> |

コマンドは断りが無い限り **<project>** の中（プロジェクトルート）で実行します。 `--root .` が付いているのはそのためです。

### 1. skill の配置

skills リポジトリの中で、次の 2 つをコピーします（**cwd は skills リポジトリの中**）。

```
cd <skills>
cp -r legacy-reverse <project>/.claude/skills/
cp -r legacy-reverse/skills/* <project>/.claude/skills/
```

PowerShell の場合:

```
cd <skills>
Copy-Item -Recurse legacy-reverse <project>\.claude\skills\
Copy-Item -Recurse legacy-reverse\skills\* <project>\.claude\skills\
```

**2 行必要な理由:** 1 行目は skill 本体一式（スクリプト・hook・テンプレート）を **<LR>** に置きます。2 行目はフェーズ別 skill を `.claude/skills/` の**直下**にも置きます——ClaudeCode はここを見て `/legacy-1-spec` などのコマンドを認識するためです。中身が重複しますが、これで正しい状態です。

配置後、Claude Code を開き直すと `/legacy-reverse` が使えるようになります。

## 2. hook の登録（必須）

legacy-reverse/hooks/settings-example.json の内容を <project>/.claude/settings.json にマージします。2つの安全装置が入っており、どちらもプロンプト（お願い）ではなく仕組みで事故を防ぎます。

Table 3

| hook                        | 効くとき               | 何を防ぐか                                 |
|-----------------------------|--------------------|---------------------------------------|
| guard_tests.py (PreToolUse) | ④⑤フェーズ中の tests/ 編集 | AI がテストを書き換えて通す事故                     |
| guard_json.py (PostToolUse) | .json を編集した直後      | AI が functions.json 等を壊してパイプラインが止まる事故 |

guard\_json.py は、AI が JSON を編集した直後にその場で構文を検証し、壊れていれば **AI 本人に差し戻して直させます**（「1 行 37 列: 末尾に余分なカンマ」のように位置つきで返します）。data/functions.json は全スクリプトが毎回読む正データなので、ここが壊れると無人バッチが途中で止まります。①～⑦のどのフェーズでも効きます。

すでに運用中のプロジェクトに後から入れる場合も、settings.json に PostToolUse のブロックを追記するだけです（既存の PreToolUse はそのまま残してください）。

## 3. MCP サーバの登録（推奨）

**MCP (Model Context Protocol)** は、AI に外部のプログラムを「道具」として渡すための仕組みです。ここで登録するのは、このパイプラインの機械処理（台帳の読み書き・機械レビュー・テスト実行など）を AI から呼べるようにした小さなサーバで、実体は skills リポジトリの中の Python スクリプト 1 本です。

<project>/.mcp.json に次を書きます（**ファイルが無ければ新規作成**）。

```
{
  "mcpServers": {
    "legacy-reverse": {
      "command": "python",
      "args": ["C:/work_space/skills/mcp-servers/legacy-reverse-mcp/server.py"]
    }
  }
}
```

- args のパスは <skills> の絶対パスに置き換えてください（上は例）。<LR> ではなく **skills** リポジトリ側を指します——だからセットアップ後も リポジトリを消してはいけません
- 書いたら Claude Code を開き直します

**登録すると何が変わるか:** 登録しなくても動作は同じですが、登録すると AI がスクリプトを「シェルコマンド」ではなく「型の決まった道具」として呼べるようになります。その結果、**実行のたびに出る許可確認（このコマンドを実行していいですか、のダイアログ）が減り**、引用符やパスの取り違いによる失敗も起きにくくなります。

## 4. ツール類

- **Quarto**（必須。進捗サイトの HTML 生成に使います） — [quarto.org](https://quarto.org) のインストーラで入れてください。quarto --version が通れば OK です。

管理者権限を使いたくない場合は、skills リポジトリ同梱のヘルパーで ホームディレクトリ配下にポータブル導入もできます (PATH も変更しません) : `python <skills>/quarto-typst-pdf/scripts/qtpdf.py install`

- **Python パッケージ**: `pip install pytest sphinx sphinx-rtd-theme` (⑦では追加で `radon ruff bandit pip-audit`)

**合本 PDF (関数仕様書などを 1 冊にまとめた PDF) も出す場合のみ、追加で 2 つ:**

- skills リポジトリの `quarto-typst-pdf/` を `<project>/.claude/skills/` にコピー (pdf\_book.py が <LR> の隣を探すため。日常の HTML 生成には不要です)
- **Chromium 系ブラウザ** (Chrome/Edge 等) — PDF に Mermaid 図を焼き込むのに使います (HTML 側はブラウザが描くので影響ありません)

## 5. 開始

Claude Code で `/legacy-reverse` を実行すると、セットアップ状態を確認して 次の一手を案内します。新規なら `/legacy-0-analyze` へ。

`/legacy-reverse` が候補に出てこない場合は、Claude Code を開き直してください (skill の配置は起動時に読み込まれます)。

## フェーズ別ガイドー各フェーズで人がやること

すべてのフェーズは**人がスラッシュコマンドで起動**します。AI が勝手に次のフェーズへ進むことはありません。

Table 4

| フェーズ     | コマンド                                                 | 人がやること                                                                                                                                                                                       |
|----------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ① 解析     | <code>/legacy-0-analyze</code>                       | レガシーの場所・言語・新パッケージ名を答える。規約（conventions.md）を <b>あなたが記入</b> して確定（AI は質問リストと記入例を出すだけ。Fortran と C/C++ の関数列挙は静的解析プログラムが行い、AI は説明の充填と抽出レポートの確認のみ）。0 割などの扱いを決める（後述）。必要なら仕様書の項目立て（docs/templates/）を編集 |
| ② 変数辞書   | <code>/legacy-0-dict</code>                          | 変数の意味を承認する（一括承認と、1 件ずつの確認）。詳しくは「変数辞書と例外ポリシー」の章                                                                                                                                               |
| ③ 仕様書    | <code>/legacy-1-spec F-xxxx</code> （「まとめて 10 件」でバッチ） | レビューして OK を出す（reviewed 化）。🟡🔴 項目の質問に答える。バッチ時は一斉レビュー表でまとめて                                                                                                                                     |
| ④ テスト仕様  | <code>/legacy-2-testspec F-xxxx</code>               | ケース一覧と質問を見て承認（approved 化）。期待値の不明点に答える                                                                                                                                                        |
| ⑤ テストコード | <code>/legacy-3-testcode F-xxxx</code>               | 基本は見守り（完了時に freeze される）                                                                                                                                                                      |
| ⑥ 実装     | <code>/legacy-4-impl F-xxxx</code>                   | 基本は見守り。spec-gap ISSUE が来たら①改訂を指示                                                                                                                                                             |
| ⑦ テスト    | <code>/legacy-5-test F-xxxx</code>                   | fail 時の裁定（後述）。pass なら WBS で✅を確認                                                                                                                                                              |
| ⑧ 完了検証   | <code>/legacy-6-check</code>                         | 全関数完了後に 1 回。不備リストが出たら該当フェーズへ差し戻し                                                                                                                                                             |
| ⑨ 分析・改善  | <code>/legacy-7-analyze</code>                       | 代表ワークロードを教える。施策票を承認する                                                                                                                                                                        |

「次にどの関数をやるべきか」は WBS の推奨着手順（依存の葉から）に従います。迷ったら `/legacy-reverse` に聞けば next を提案します。

## 関数リストはあとから調整できる（追加・対象外）

①で作った関数リストは固定ではありません。進めている途中で「この関数も追加して」「この関数は移植しない」と AI に指示すれば調整できます：

- **追加**：「XXX という関数を追加して（legacy/tax.f の 120～240 行）」→採番され、骨子と WBS に載り、以後は他の関数と同じ①～⑤を回せます

- **対象外:** 「F-0123 は使われていないから外して」 → ①～⑥・WBS の対象から外れます。消えるのではなく、WBS 末尾の「対象外の関数」に理由つきで残ります（監査で「なぜ移植していないのか」に答えられる形）。あとで「F-0123 を対象に戻して」と言えば復帰もできます

functions.json からエントリを手で消してはいけません。⑩の再解析でソースから別 ID として復活してしまい、作りかけの仕様書との紐付けが切れます。必ず上の操作で。

## ①②の承認とは何をするとか

AI は承認を求めるとき、**変更点サマリ・Confidence**（● 確認済/● 推測/● 仮定）の内訳・未回答の質問一覧をチャットに提示します。あなたは:

- 内容に問題なければ「OK」と返す → AI が status を更新する（承認直前に機械レビューを再検証）
- 質問には **Yes / No / 修正内容** で答える（AI は必ず仮説を添えて聞いてくるので、白紙から考える必要はない）
- 答えられない質問は「保留」でよい。ISSUE として残り、WBS の最上部に出続ける



## 関数仕様書: get\_rate

### 概要

割引区分コードから割引率をテーブル検索して返す。テーブル未ロード時は RATES.DAT から読み込む（フォールバック）。

### インタフェース

#### 入力

| # | 名前   | レガシー型       | 新型  | 説明                 | Confidence |
|---|------|-------------|-----|--------------------|------------|
| 1 | CODE | CHARACTER*1 | str | 割引区分コード (1文字、完全一致) |            |

#### 出力

| 名前   | レガシー型   | 新型                  | 説明                    | Confidence |
|------|---------|---------------------|-----------------------|------------|
| RATE | REAL    | float (戻り値テーブル第1要素) | 割引率。エラー時 0.0          |            |
| IERR | INTEGER | int (戻り値テーブル第2要素)   | 0=正常 1=コード無し 2=ファイル無し |            |

### グローバル状態

| 名前                                      | 読み/書き | 説明                            | Confidence |
|-----------------------------------------|-------|-------------------------------|------------|
| /RATTBL/ TCODE(10),<br>TRATE(10), NRATE | 読み書き  | 割引率テーブル (最大10件)。フォールバック時に書き込む |            |

### 参照外部ファイル

| ファイル      | 読み/書き | 用途                                                     | Confidence |
|-----------|-------|--------------------------------------------------------|------------|
| RATES.DAT | 読み    | 割引率マスタ。1行 = 区分1文字 + 率5桁<br>( (A1,F5.3) 、例:<br>A0.050 ) |            |

### 呼び出しサブルーチン

| 名前 | func-id | 用途 |
|----|---------|----|
|----|---------|----|

図 2: ①関数仕様書。機械抽出した IO 表と、機能詳細の各項目に Confidence ( ) とレガシー行番号の根拠が付く。あなたはこれをレビューして OK を出す

HTML サイトは「見せるだけ」です (画面から操作はしません)。①draft・②generated の間、仕様書ページの本文の一番上に案内パネルが出ます。機械レビューの結果 ( か、 なら理由の全文 ) と「どう返答するか」がその場を書いてあるので、「一斉レビュー表の 4 件が何なのか分からない」と別の場所を探しに行く必要はありません。返答は次の 3 チャンネルで、どれでも同じです:

- チャット: 「F-0012 OK」で承認、「F-0012 修正: ~」で修正依頼
- ファイル記入: docs/review-feedback.md に修正依頼を書く (書式はファイル冒頭。次に AI が動いたとき自動で拾われます)

#### 目次

- 概要
- インタフェース
- 機能詳細
- 副作用・例外
- 未確定事項

- **CLI:** `review_actions.py approve spec F-0012 --by 名前` / `review_actions.py request-changes spec F-0012 --by 名前 --comment ""` (承認は入口によらずサーバ側で機械レビューを再検証してから確定します。NGが残る仕様書はどの入口からも承認できません)

一斉レビュー表 (spec-review.md) の「機械レビュー」列も、**✖**の件数リンクを押せばその関数の案内パネル (理由の全文) へ直接飛びます。

ISSUE は必ず「仮説+Yes/No の問い」の形で来ます。白紙の質問は来ません。

pricing-batch リバースエンジニアリング WBS ドメイン知識 規約 ⑥完了検証 ⑦分析 新コード詳細(API) Q

## 割引率テーブルの10件打ち切りはバグ互換で維持するか

### 事象

GETRT のフォールバック読込は RATES.DAT の先頭10件で打ち切り、11件目以降を黙って無視する (`legacy/pricing.f:25`、配列サイズ10の制約)。マスタが10件を超えた場合、超過分の区分は常に「コード無し(IERR=1)」になる。エラーも警告も出ない。

### 質問 (人への問い)

仮説: これは配列サイズ由来の制約であり意図した仕様ではないが、移植ではバグ互換で10件打ち切りを維持する (現行マスタは10件以内で運用されていると推測。上限拡張は挙動変更になるため⑦以降で検討)。

問い: この理解で正しいですか? (Yes / No / 修正)

### 回答 (人が記入)

- 回答: Yes。バグ互換で10件打ち切りを維持。上限拡張は⑦以降で検討。
- 回答者 / 日付: havealinch / 2026-07-25

### 反映

| 反映先                  | 内容                               |
|----------------------|----------------------------------|
| docs/specs/F-0001.md | SPEC-F0001-02 の10件打ち切りを正式仕様として確定 |

目次

- 事象
- 質問 (人への問い)
- 回答 (人が記入)
- 反映

図 3: ISSUE の例。AI の仮説に Yes/No/修正 で答えるだけでよい。回答はドメイン知識集に転記され、同じ質問は二度と来ない

## 実行はチャットか CLI から (1 関数ずつ／まとめて)

実行の入口は 2 つです。1 関数ずつ様子を見ながら進めるならチャット (`/legacy-1-spec F-0012` のようにフェーズ skill を呼ぶ)、まとめて無人で進めるなら CLI の `pipeline.py`:

```
python <LR>/scripts/pipeline.py spec --root . # ①だけを全件 draft まで
python <LR>/scripts/pipeline.py run --root . # ①～⑤を工程横断で回し続ける
```

- `run` は全関数を走査して「次に着手できる工程」を 1 件ずつ実行し、終わるたびに再走査します。③が終われば同じ関数の④、承認が下りれば次の工程、と自動的に進みます。承認待ち (①②) と裁定待ち (⑤) はスキップして先へ進むので、あなたは並行して承認・裁定だけしていけばよい (承認した分は次の走査で拾われます)
- 進行は関数ごとです。①が全件終わるのを待つ必要はありません——ある関数の①を承認すれば、その関数はすぐ②へ進めます。「①をまとめて流して一斉レビュー」は効率のための運用であって、順序の制約ではありません

- 修正依頼 (pending) や機械レビュー NG が残っている①②は、run が自動で「AI の自己修正」として再実行対象に拾います（チャットで個別に頼んでも同じ）
- fail した⑤のうち (a)実装バグは、1 回の実行の中で AI が「修正→再実験」を回して pass か裁定待ちまで進みます。裁定待ち (🛑) になった関数は、あなたが ISSUE に 回答して unblock するまで再実行対象になりません（空振り実行を防ぐ設計）
- ⑥完了検証は全関数の⑤が完了すると WBS トップに案内が出ます (ledger check または /legacy-6-check。LLM を呼ばないので数十秒で終わります)
- ⑦分析は⑥が pass してから /legacy-7-analyze をチャットで呼びます。cProfile+静的解析 (radon / ruff / bandit / pip-audit) で定量評価を機械的に実測し、その実測値に基づいて AI が施策候補 (OPT-性能 / REF-保守性 / SEC-セキュリティ) を analysis.md に提案します。提案までで、コードには触りません。ルートに bench.py (代表ワークロード) を置いておくで本命の計測になります
- 排他は自動です。バッチ実行中に別のバッチは起動できません（同じ実行枠 = .legacy-reverse/run.lock を取り合う設計）。実行が異常終了してロックや「実行中」表示が残っても、次の実行時に自動で残骸と判定して回復します

## バッチ実行状況ページ — 進捗のライブ表示（見るだけ）

<http://127.0.0.1:<ポート>/pipeline.html> (WBS の「バッチ状況」からも行けます) は、無人実行 (pipeline.py) のライブ表示です。「いま何が起きているか」と「人がやるべきこと (人待ち)」がこの 1 枚に集まっています。2.5 秒ごとに自動更新されるので、開いたまま置いておけます。このページから操作はできません (開始・停止・★・承認はターミナルとチャットから。ページ上部に対応コマンドの案内が出ます)。

### バッチ実行状況 実行中

[← WBS](#) / 2.5秒ごとに自動更新 / 最終更新: 2026-08-07T08:10:55

連続実行

全部 (残り19件)

開始

停止

残タスクを実行順に自動で回します (承認・裁定待ちにはスキップして進み、このページで承認・裁定すると次の走査で拾われます)。★で順番に割り込めます

▶ F-0001 ①仕様書 calcx / legacy/input.f90・試行1

経過 1分46秒 (中央値 112s)

応答ログ (前回まで)

この件をスキップ

処理 8 / 16 件 (成功 6・失敗 2・残り 8) / レート待機 340s

75%
成功率

112s
中央値/件

20分
残り見込み

\$3.30
累計コスト (平均 \$0.410/件)

残タスク 28件

検索: F-番号・関数名・レガシーファイル

すべて

①10

②8

④1

人待ち 9

次1 ① F-0008 仕様書の作成 fluxq / legacy/output.f90

★ 優先中

今 ① F-0001 仕様書の作成 calcx / legacy/input.f90

実行中

次2 ① F-0002 仕様書の作成 interp / legacy/geometry.f90

★ 優先

次3 ① F-0003 仕様書の作成 matmul2 / legacy/output.f90

★ 優先

次4 ① F-0004 仕様書の作成 rinput / legacy/matrix.f90

★ 優先

次5 ① F-0005 仕様書の作成 wrtout / legacy/solver.f90

★ 優先

次6 ① F-0006 仕様書の作成 convrg / legacy/input.f90

★ 優先

次7 ① F-0007 仕様書の作成 eigsol / legacy/geometry.f90

★ 優先

次8 ② F-0009 テスト仕様書の作成 geomtr / legacy/geometry.f90

★ 優先

結果 直近8件

すべて

NGのみ 2

OK F-0007 ①仕様書 08:08:13・142s・\$0.41・18ターン

OK F-0006 ①仕様書 08:06:15・98s・\$0.28・12ターン

NG F-0019 ①仕様書 — 機械レビューNG 08:02:02・試行2・233s・\$0.66・31ターン

機械レビューNG (2件)

SPEC-F001902: (確認済) なのに有効な根拠引用 (file:lines) がない

「副作用・例外」が空欄 (なければ「なし」と明記する規則)

応答ログ

★ 優先して再実行

▶ 応答の末尾

OK F-0005 ①仕様書 07:59:41・121s・\$0.35・15ターン

OK F-0023 ②テスト仕様書 07:52:29・87s・\$0.22・10ターン

NG F-0028 ⑤テスト — 検証NG 07:52:29・試行3・305s・\$0.88・42ターン

検証NG (1件)

test\_edge\_zero: expected 0.0, got nan

応答ログ

★ 優先して再実行

▶ 応答の末尾

OK F-0004 ①仕様書 07:50:19・110s・\$0.31・14ターン

OK F-0022 ②テスト仕様書 07:48:43・76s・\$0.19・9ターン

11

図 4: バッチ実行状況ページ。実行中の 1 件・全体進捗・指標カード、下段は左が残タスク（実行順・検索・人待ち）、右が結果（NG 理由を開いた状態で表示）

① **実行中の 1 件（緑のバー）** — いま動いている関数・工程・試行回数と、経過時間が出ます。バーは**その工程の中央値を 100% とした進み具合**で、中央値を超えると黄色、2 倍を超えると赤くなり「長引いています」と出ます。異常に気づくための表示です。「応答ログ（前回まで）」でこの関数のエージェント応答の全文が読めます（実行中に読めるのは前回の試行まで）。

② **全体進捗と指標カード** — 処理件数・成功率・1 件あたりの所要（中央値）・残り見込み（ETA）・累計コスト。**最初の数件でこれを見てから、夜間に何件回すかを決めてください。**

③ **残タスク（左下）** — 連続実行が次に何をやるかが実行順に並びます。「今 / 次 1 / 次 2 …」の番号がそのまま実行順です。

- **検索**（F-番号・関数名・レガシーファイル名）と工程チップ（①②③④⑤・人待ち）で絞り込みます。2000 関数でも既定表示は先頭 9 件だけなので、画面が重くなりません
- 順番に割り込みたいときはターミナルから `pipeline.py priority F-xxxx`（実行中の 1 件が終わり次第、先頭に割り込みます。★印が付きます）
- **人待ちタブ**: 承認待ち・裁定待ちがここに集まります。バッチが自動でスキップし続ける = **人待ちこそが本当のボトルネック**です。「開く」で該当ページへ行くと、ページ先頭の案内パネルに返答方法（チャット / ファイル記入 / CLI）が出ています

④ **結果（右下）** — 直近の実行結果です。OK 行は 1 行、NG 行は**理由が開いた状態**で表示されます（機械レビューの指摘は箇条書きで全文）。「応答ログ」でエージェント応答の全文が読めます（失敗原因の一次情報はここにしかありません）。「NG のみ」タブで失敗だけを拾えます。

**失敗してスキップされた関数はどうなるか**: 失敗した(関数, 工程)は そのバッチの中でだけ再試行対象から外れます（同じ関数で失敗し続けて全体が止まらないようにするため）。`pipeline.py priority F-xxxx` で拾い直せますし、何もしなくてもバッチを再度起動すれば普通に対象へ戻ります。成果物の状態がすべての正なので、失敗で進捗が失われることはありません。なお**連続 3 件失敗するとバッチ全体が安全停止**します（API キー切れなどの環境異常で全件を失敗として消化してしまうのを防ぐためです）。

## ①はまとめて進めて、まとめてレビューできる

1 件ずつ「実行→レビュー」を回す必要はありません。「仕様書をまとめて 10 件進めて」と頼むと、AI が推奨着手順に沿って複数関数を機械レビュー通過済みの draft まで連続で仕上げ、**一斉レビュー表**（`docs/spec-review.md`。WBS の「要対応」からリンク）にまとめて持ってきます。

## ① 仕様書 一斉レビュー

公開  
2026年8月7日

レビュー待ち (draft) : 10 件。概要と Confidence を見て、そのまま承認するか、関数名リンクで仕様書全文を確認してください。

承認・修正依頼は各行のボタンで完結します (チャットで「全部OK」／「F-xxxx は修正: ~」でも可)。✖ は AI が自己修正するまで承認できません。

| 関数                     | 概要                                      | Confidence | 機械レビュー | 未確定 (ISSUE) | 操作                                         |
|------------------------|-----------------------------------------|------------|--------|-------------|--------------------------------------------|
| <a href="#">bndry</a>  | BNDRY の計算処理。反復計算の1ステップ分を実行し収束判定値を更新する。  | 2 ●        | ✓      | —           | <a href="#">承認</a> <a href="#">修正依頼...</a> |
| <a href="#">solve1</a> | SOLVE1 の計算処理。反復計算の1ステップ分を実行し収束判定値を更新する。 | 2 ●        | ✓      | —           | <a href="#">承認</a> <a href="#">修正依頼...</a> |
| <a href="#">preprc</a> | PREPRC の計算処理。反復計算の1ステップ分を実行し収束判定値を更新する。 | 2 ●        | ✓      | —           | <a href="#">承認</a> <a href="#">修正依頼...</a> |
| <a href="#">postpr</a> | POSTPR の計算処理。反復計算の1ステップ分を実行し収束判定値を更新する。 | 2 ●        | ✓      | —           | <a href="#">承認</a> <a href="#">修正依頼...</a> |
| <a href="#">normv</a>  | NORMV の計算処理。反復計算の1ステップ分を実行し収束判定値を更新する。  | 2 ●        | ✓      | —           | <a href="#">承認</a> <a href="#">修正依頼...</a> |

図 5: ① 仕様書 一斉レビュー。承認できる行が上に並び、チップと検索で絞り込める。関数名リンクから仕様書全文へ行ける

表の読み方と使い方:

- **並び順が優先度**です。上から「そのまま承認できる」→「ISSUE (あなたへの質問) がある」→「AI 修正待ち (機械レビュー NG)」の順に並びます。上から片付ければ、判断が要らないものから消化できます
- **チップと検索**で絞り込めます。「承認できる N」を押せば、いま承認可能な行だけが残ります。件数はチップに出るので、残り作業量がひと目で分かります
- **概要**は仕様書の冒頭 1 行です。これで判断がつくものはその場で承認し、怪しいものだけ関数名リンクから全文を読む、という使い分けができます
- **Confidence** (● 確認済 / ● 推測 / ● 仮定) の内訳が出ます。● ● が多い関数は中身を見るべき、という目印です
- **状態列**に「承認できる」「AI 修正待ち」が出ます。返答はチャットで「全部 OK」／「F-0012,F-0034 は OK」／「F-xxxx は修正: ~」とまとめて返すのがいちばん速い運用です (CLI の `review_actions.py approve` を 1 件ずつ叩いても、`review-feedback.md` に書いても同じ)。どの入口でも承認直前に機械レビューを再検証してから確定するので、NG 付きの仕様書が承認されることはありません

補足:

- 気に入らない仕様書は**何度でも書き直しを頼めます** (draft のうちは自由。reviewed 済みを書き直す場合は②の再承認が必要になる旨を先に確認されます)
- 承認が人であることは変わりません。粒度が「1 件ずつ」か「まとめて」かの違いだけです

**数百~2000 件の無人実行は CLI ドライバで行います。** `spec` (①専用) は仕様書だけを全件 draft まで流す用、`run` (①~⑤) は工程横断に進める用です (★優先も反映されます)。`run` に `--only` を付けると工程を限定でき、「②だけ全件」のような工程単位のバッチにもなります。チャットの AI と違いトークン上限がなく (1 関数ごとに新しい AI プロセスを起動する方式)、夜間に回して翌朝一斉レビュー表を見る、という運用ができます。途中で止めても同じコマンドで続きから再開します。

```
python <LR>/scripts/pipeline.py run --root . --dry-run # まず対象と実行順を確認
python <LR>/scripts/pipeline.py run --root . --max-funcs 20 # 最初は少なく試す
```



```
python <LR>/scripts/pipeline.py run --root . # ①～⑤を回し続ける
python <LR>/scripts/pipeline.py run --root . --only testspec # ②だけ全件流したいとき
python <LR>/scripts/pipeline.py spec --root . # ①だけ全件流したいとき
```

★優先（順番の割り込み）はターミナルから（実行中のバッチにも「いまの1件が終わり次第、先頭に割り込む」形で即座に効きます）：

```
python <LR>/scripts/pipeline.py priority F-0012 # F-0012 を次に割り込ませる
python <LR>/scripts/pipeline.py priority # いまの★一覧を見る
python <LR>/scripts/pipeline.py priority F-0012 --off # 解除（--clear で全解除）
```

始める前に、ツールの実行許可を通しておいてください。無人実行は許可ダイアログに答えられないので、許可待ちで止まると朝まで無駄になります。方法は2つです。

- <project>/`.claude/settings.json` の `permissions.allow` に、①が使うツール（Read / Write / Edit と、スクリプト実行の Bash）を並べておく
- 信頼できる閉じた環境なら `--skip-permissions` を付けて起動する（内部的に `--dangerously-skip-permissions` を渡します。共有マシンでは使わないでください）

claude が PowerShell からしか見つからない環境では、`(Get-Command claude).Source` で実体のパスを調べ、`--claude-cmd "<そのパス>"` で明示すると確実です。

実行中の様子は前述のバッチ実行状況ページ（/pipeline.html）で見られます。URL はバッチ起動時にターミナルにも表示されます。

## ⑤で fail したときの裁定

テストが落ちたとき、原因は3種類あります。(a) だけは AI が自走し、(b)(c) はあなたの承認が要ります。

Table 5

| 分類           | 意味           | あなたの操作                      |
|--------------|--------------|-----------------------------|
| (a) 実装バグ     | 実装が①を満たしていない | 何もしない（AI が④修正→⑤再実行を回す）      |
| (b) テストコードバグ | ③が②とズレている    | ISSUE の証拠を見て承認 → ③再生成を指示    |
| (c) 仕様が誤り    | ②①自体が間違い     | ISSUE で裁定 → ①改訂を指示（伝搬は自動検知） |

(a) のループは3回で自動停止し、triage ISSUE が起票されて WBS に ➊ が出ます。このときの再開手順:

1. ISSUE を読んで (a)/(b)/(c) を裁定し、回答欄に記入（またはチャットで回答。1 コマンドで済ませたいときは `review_actions.py adjudicate F-xxxx --issue ISSUE-xxx --by 名前 --comment "回答"` ——記入から unblock まで一気に行います）
2. ファイル記入・チャットの場合は、AI が反映（applied）したら「F-xxxx をアンブロックして」と言う
3. `/legacy-5-test F-xxxx`（または `pipeline.py run`）を再トリガ（試行回数は1からやり直し）

裁定待ちの関数の仕様書ページには、ISSUE の質問と上記の返答方法が案内パネルとして表示されます（チャットに戻らなくても、何をすればよいかはページ上で分かります）。

## テスト結果: get\_rate

### サマリ

目次

[サマリ](#)[ケース別結果](#)[失敗詳細](#)[試行履歴](#)

| 仕様ケース数 | 実装済み | 実装率  | 成功 | 失敗 | スキップ | 実行時間  |
|--------|------|------|----|----|------|-------|
| 6      | 6    | 100% | 6  | 0  | 0    | 0.03s |

### ケース別結果

| ケースID        | 内容                                  | 分類                  | 実装 | 結果 | 時間     | 詳細 |
|--------------|-------------------------------------|---------------------|----|----|--------|----|
| F0001-TC-001 | <a href="#">ロード済みテーブルからのヒット</a>     | 正常系                 | ✓  | ✓  | 0.0s   |    |
| F0001-TC-002 | <a href="#">未ロード時のフォールバック読み込み</a>   | 外部ファイルI/O / グローバル状態 | ✓  | ✓  | 0.013s |    |
| F0001-TC-003 | <a href="#">10件打ち切り (11件目以降は無視)</a> | 境界値                 | ✓  | ✓  | 0.015s |    |
| F0001-TC-004 | <a href="#">コード不一致</a>              | 異常系                 | ✓  | ✓  | 0.0s   |    |
| F0001-TC-005 | <a href="#">マスタファイル無し</a>           | 異常系 / 外部ファイルI/O     | ✓  | ✓  | 0.0s   |    |
| F0001-TC-006 | <a href="#">空ファイル</a>               | 境界値                 | ✓  | ✓  | 0.001s |    |

### 失敗詳細

なし

### 試行履歴

図 6: ⑤テスト結果報告書。実装率（②のケースが③に全部実装されたか）と、ケースごとの内容・分類・結果が 1 枚で分かる。内容列は②の該当ケース定義へのリンク

変数辞書と例外ポリシー — ①より先にやること

⑨の解析が終わったあと、①の仕様書に入る前に片付ける仕事が2つあります。どちらも**あとから直す**と**全仕様書に波及する**種類の決めごとなので、先にやります。

## なぜ辞書が先なのか

レガシーの変数名は `R8TBL・WKA・IFLG` のように、それだけでは意味が分かりません。そして同じ変数が `COMMON` 経由で 10 も 20 もの関数に顔を出します。

語義を決めないまま仕様書を書くと、**同じ実体に関数ごとに違う説明が付きます**。「税率テーブル」「レート配列」「R8TBL (用途不明)」——後で1つに揃えようとすると、既にレビュー済みの仕様書を全部開き直すことになります。

そこで、「**1 変数 = 1 つの意味**」をあなたが先に確定させてから①へ渡します。確定した意味は仕様書の入出力表へ自動で転記され (年間税率(比率) [V-0001] の形)、①はそれを書き換えません。

既定では、**意味が未承認の変数を含む関数は①に進めません** (AI が「先に辞書を」と言ってくるのはこの仕組みのためです)。急ぐときは解除もできます (後述)。

## 変数辞書の作られ方

1. **機械が変数をまとめます**。COMMON の同じ位置・呼び出しの実引数と仮引数・EQUIVALENCE・同じ関数内の同名——この4つの**機械的な根拠**だけで「同じ実体」を判定します。名前が同じでも根拠がなければ別物として扱い、「同名別義」として並べて表示します (取り違えを防ぐため)
2. **機械が根拠を集めます**。宣言行のコメント、`WRITE/FORMAT` の見出し文字列、`DATA/PARAMETER` の初期値、使われている式——これが AI に渡す唯一の材料です
3. **AI が意味を書きます**。AI はレガシー全文を読みません。**上で集めた根拠しか読めず、引用も根拠の ID でしか行えません**。存在しない ID を書けば機械が全件差し戻します。根拠から意味を決められないものは「不明」と返すのが正しい動作です
4. **機械が確からしさ (rank) を判定します**。AI の自己申告ではありません:

Table 6

| rank | 意味                                        |
|------|-------------------------------------------|
| A    | コメント・FORMAT の見出し・初期値という <b>明示的な根拠</b> がある |
| B    | ドメイン知識、または既に承認済みの変数とつながっている               |
| C    | 使われ方 (式) だけからの推定                          |
| D    | 根拠なし → 採用せず、あなたの確認待ちに回る                   |

5. **あなたが承認します**。ここだけが人の仕事です

## 承認のしかた

「変数辞書」ページ (ナビバーに自動で出ます) で語義と根拠を読みます。並び順は「**多くの関数で使われているもの**」から、「**確認が要るもの**」を先に——つまり上から順に確認すれば、効果の大きいものから終わります。未承認が残る間は、ページ先頭に承認方法の案内が出ます。

承認はチャットか CLI で行います (どちらも同格・記録される内容は同じ) :

- チャット: 「V-0001 と V-0002 を承認して」「V-0003 は年間税率に直して承認して」
- CLI: `variables.py approve V-0001,V-0002 --by 名前` / `variables.py revise V-0003 --desc "年間税率" --by 名前`



rank A/B は根拠が明示的なのでまとめて承認で構いません（AI が一括承認候補として 提示します）。rank C/D は 1 件ずつ、意味を確定させながら承認します（revise は修正と承認を同時に確定させます）。1 回の承認で、その変数が出てくる 全関数に効きます。

承認後は仕様書の入出力表への転記（propagate）まで続けて実行されるので、AI がバッチで解釈を回している最中でも、あなたは並行して承認を進められます。

分からない変数は保留で構いません。承認しない限り、その変数を使う関数の①は始まらないだけです。どうしても先に進めたいときは「辞書のゲートを解除して」と言えば①に進めます（あとから辞書を承認したときに、仕様書との食い違いは機械が一覧にしてくれます）。

## あとから変数が増えたとき

レガシーに追加開発が入って再解析しても、やり直しにはなりません。

- 一度承認した意味はそのまま残ります（根拠が変わっていない限り）
- 出現箇所が増えただけなら承認は維持され、「出現が増えました」という印が付きます
- 「別物だと思っていた 2 つが実は同じだった（逆も）」というまとめり方が変わった場合だけ、もう一度あなたの確認に戻ります

既にレビュー済みの仕様書が、辞書の改訂で勝手に書き換わることはありません。食い違いの候補が一覧（「辞書と仕様書の矛盾候補」ページ）に出るので、直すかどうかはあなたが決めます。WBS の要対応にも「△辞書 stale」として出ます。

## 例外ポリシー — 0 割をどう扱うか

Fortran は 0 で割っても Inf（無限大）を作ってそのまま走り続けます。Python はそこで止まります（ZeroDivisionError）。同じことが平方根の負値、対数の 0・負値、配列の範囲外にも起きます。

つまり「レガシーどおりに書いた」だけでは、動きが変わります。⑨の解析はこうした箇所（0 割・SQRT/LOG の定義域・変数添字）を機械で洗い出しており、扱いを決めないと①へ進めません。

## exception-queue への答え方

AI は「例外ポリシー 未決定キュー」というページを出してきます。そこには種類ごとにまとめて「0 割が N 箇所あります。既定をどうしますか」と書かれています。1 箇所ずつ聞かれるのではなく、種類ごとに 1 回答えれば全箇所に効きます。

答えの選択肢は 5 つです：

Table 7

| 答え                             | 意味                    | 使いどころ                                 |
|--------------------------------|-----------------------|---------------------------------------|
| <code>guard_raise</code>       | チェックして例外を出す           | 迷ったらこれ。異常を握り潰さない                      |
| <code>guard_value</code>       | 代わりの値を返して続行           | 業務上「0 のときは 0 扱い」等が決まっている場合（値も伝えてください） |
| <code>legacy_preserve</code>   | Inf/NaN のまま流す（レガシー再現） | 下流が Inf を前提にしている場合                    |
| <code>detect_only</code>       | 仕様書に書くだけ              | 実際には起こり得ないが記録は残したい場合                  |
| <code>caller_guarantees</code> | 呼び出し元が保証している          | なぜ保証されるのかも伝えてください（根拠が要ります）            |

「種類ごとの既定は `guard_raise`、ただし F-0012 のここだけは呼び出し元が保証」のような答え方もできます（**個別の指定が全体の既定に勝ちます**）。

決めた内容は「例外ポリシー登録簿」に承認者と日付つきで積み上がり、以後の関数にも自動で適用されます。①の仕様書には「どの箇所に・どのポリシーを適用し・新実装ではどう振る舞うか」の表が必ず載り、②のテスト仕様には**その境界ケース（0・0 近傍・負値）のテストが必ず作られます**——書き忘れは機械が検知して差し戻します。

## 品質ゲート — AI の誤りを機械で検知する仕組み

AI がハルシネーション（存在しない根拠の引用・仕様の捏造）や手抜き（記入の省略・スタブ実装）をしても、**人のレビューに届く前に機械が検知して差し戻す仕組み**が各フェーズに入っています。あなたが受け取る成果物は、以下をすべて通過済みです。

Table 8

| フェーズ | ゲート                                                  | 機械が検知するもの                                                                                                           |
|------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| ①    | 抽出器内蔵の完全性突合                                          | 2 系統のカウント差分（関数の数え漏れ）。結果は <code>data/extract-report.json</code>                                                      |
| ① 辞書 | <code>variables.py</code> <code>verify-interp</code> | 対象変数の書き漏らし・書きすぎ、 <b>存在しない根拠 ID の引用（捏造）</b> 、根拠なしの意味づけ（採用せず人へ回す）                                                     |
| ①    | <code>review_checks.py</code> <code>spec</code>      | <b>実在しないレガシー行への引用</b> 、●（確認済）なのに根拠なし、記入の省略（プレースホルダ残存）、必須節の欠落、レガシー原本の変更、 <b>0 割等の検討漏れ・存在しないポリシー ID の引用・未決定のまま仕様化</b> |
| ②    | <code>review_checks.py</code> <code>testspec</code>  | ● 仕様項目のテストケース漏れ、①に <b>存在しない仕様 ID の参照（捏造）</b> 、宙に浮いたケース参照、期待値根拠の規定外表記、 <b>0 割等の境界ケース漏れ</b>                           |
| ③    | マーカー突合                                               | ②のケース ID とテスト関数の過不足                                                                                                 |
| ④    | <code>check_stubs.py</code>                          | 空実装・NotImplementedError・TODO/FIXME（スタブ化）                                                                            |
| ⑤    | <code>ledger</code> <code>verify</code>              | ②の陳腐化・freeze 後のテスト改変・裁定待ちの見落とし                                                                                      |
| ⑥    | <code>review_checks.py</code> <code>all</code>       | 上記①②の全関数総点検                                                                                                         |

したがって**あなたのレビューは「意味が正しいか」に集中できます**。「引用された行は実在するか」「トレサビリティは揃っているか」の照合作業は不要です。機械で取れないのは「式は書いてあるがレガシーの意図と違う」という意味レベルのずれで、これは①のレビューと⑤の突き合わせ（独立に作ったテスト vs 実装）が受け持ちます。

大規模プロジェクト（数百～2000 関数）と中断・再開

WBS は自動でダッシュボードになる

200 関数を超えると、WBSのトップページは全関数表をやめてダッシュボードに切り替わります（進捗サマリ・要対応・あなたへの質問・次の一手・レガシーファイル別の進捗表）。全関数の明細はファイル別のサブページに分かれ、リンクで行き来できます。2000 関数でも「いま何が問題で、次に何をやるか」はトップの 1 画面で分かります。

bigproj リバースエンジニアリング WBS    ドメイン知識    規約    ⑥完了検証    ⑦分析    新コード詳細(API)

### bigproj リバースエンジニアリング WBS

公開  
2026年8月3日

#### 進捗サマリ

| 関数数 | ①spec | ②test-spec | ③test-code | ④impl | ⑤test |
|-----|-------|------------|------------|-------|-------|
| 300 | 0/300 | 0/300      | 0/300      | 0/300 | 0/300 |

#### コールグラフ

関数 300 件のため図は省略（依存は下表の「依存」列を参照）。

#### Open ISSUES（要人間判断）

なし 🚩

#### 次の一手（推奨着手順 上位10）

残り 290 件は関数一覧を参照

| #  | 関数                      | 次フェーズ |
|----|-------------------------|-------|
| 1  | <a href="#">sub0001</a> | ①spec |
| 2  | <a href="#">sub0002</a> | ①spec |
| 3  | <a href="#">sub0003</a> | ①spec |
| 4  | <a href="#">sub0004</a> | ①spec |
| 5  | <a href="#">sub0005</a> | ①spec |
| 6  | <a href="#">sub0006</a> | ①spec |
| 7  | <a href="#">sub0007</a> | ①spec |
| 8  | <a href="#">sub0008</a> | ①spec |
| 9  | <a href="#">sub0009</a> | ①spec |
| 10 | <a href="#">sub0010</a> | ①spec |

#### 関数一覧（300 件・レガシーファイル別）

| レガシーファイル                       | 関数数 | ①spec | ②test-spec | ③test-code | ④impl | ⑤test | 要対応 |
|--------------------------------|-----|-------|------------|------------|-------|-------|-----|
| <a href="#">legacy/mod01.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod02.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod03.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod04.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod05.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod06.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod07.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod08.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod09.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |
| <a href="#">legacy/mod10.f</a> | 30  | 0/30  | 0/30       | 0/30       | 0/30  | 0/30  | —   |

目次

[進捗サマリ](#)  
[コールグラフ](#)  
[Open ISSUES（要人間判断）](#)  
[次の一手（推奨着手順 上位10）](#)  
[関数一覧（300 件・レガシーファイル別）](#)

図 7: 大規模モードの WBS（300 関数の例）。トップは進捗サマリ・次の一手・ファイル別進捗のダッシュボードになり、全関数の明細はファイル別サブページへ分割される

20

## 中断しても 1 からやり直しにならない

進捗の正はすべてファイル（functions.json・ledger.json・成果物のフロントマター）にあり、AI の会話コンテキストには置きません。セッションが切れても、日をまたいても:

- **再開はいつでも /legacy-reverse を打つだけ**。要約（数行）と着手可能リストを 読んで次の一手を提案してきます。全関数の再列挙はしません
- ①の解析を途中でやり直しても、抽出器は**マージ動作**（関数 ID は不変・手修正は保持・新規だけ追加）なので、採番のやり直しや成果物の宙吊りは起きません
- 「どこまで進んだか分からなくなった」ときは WBS を開くのが最短です。🚫・⚠️・❌ が「要対応」  
として最上部にまとまっています

## WBS サイト

進捗はすべて HTML サイトで確認します（各フェーズの最後に自動更新されます）。次のコマンドで自分の PC のローカルホストに立ち上がり、ブラウザが開きます。

```
python <LR>/scripts/serve_site.py --root .
```

- 表示される URL (<http://127.0.0.1:8xxx/>) は**プロジェクトごとに固定**なのでブックマークできません。複数の移植プロジェクトを同時に立ち上げてもポートはぶつかりません
- 自分の PC の中だけで配信します（社内 LAN にも出ません）。誰かに見せたいときは後述の「レビューへの配布」を使ってください
- 立てたまま作業して構いません。フェーズが進んだらブラウザを更新するだけで最新になります
- `--watch` を付けると、`docs/` のファイルを編集した時点で自動的に作り直されます（ドメイン知識や ISSUE 回答を書きながら見るとき用）
- **トップ = WBS**: 進捗サマリ、コールグラフ図（緑=完了/黄=着手中/灰=未着手/赤=判断待ち。ノードをクリックで仕様書へ）、Open ISSUE（あなたへの質問が常に最上部）、関数一覧（各 ☒ から成果物へリンク）、⑦改善イタレーションの達成状況
- ナビバー: ドメイン知識 / 規約 / **変数辞書**（作られていれば自動で出ます。未承認があるときは承認方法の案内付き） / ⑥完了検証 / ⑦分析 / 新コード詳細(API) / **バッチ状況**（無人実行のライブ表示。前述） / **マニュアル**（本書。サイトに同梱されるのでレビューアーに配った EXE から読めます）
- API ページ (Sphinx) : 関数一覧 → 個別ページ。Spec: 行で仕様書へ、トレース対応表で func-id ↔ 新関数 ↔ レガシー名 ↔ 仕様書が相互に辿れる



図 8: 新コード詳細(API)。docstring から自動生成され、関数一覧→個別ページ、トレース対応表から仕様書へ渡れる

## レビュアーへの配布（実行するだけの EXE）

進捗を上司やレビュアーに見せるのに、サーバを用意したり社内公開したりする必要はありません。サイトを丸ごと入れた実行ファイルを渡し、**各自が自分のローカルホストで開く形**にします。

```
pip install pyinstaller # 初回のみ
python <LR>/scripts/build_viewer.py --root . # → dist/<プロジェクト>-wbs.exe
```

渡された人はダブルクリックするだけです（黒い画面が出たまま、ブラウザに WBS が開きます。見終わったら黒い画面を閉じるか Ctrl+C）。

- **相手の PC に Python も Quarto も要りません。** ファイルは 10MB 程度
- ネットワークに繋がっていない PC でも開けます（外部フォントの参照は除去済み）
- 中身は**ビルドした時点のスナップショット**です。進捗が動いたら作り直して渡し直してください
- Windows 用の `.exe` は Windows 上でビルドしてください（他 OS からは作れません）
- 会社の設定で「発行元不明の実行ファイル」が止められる場合は `--onedir` を付けるとフォルダ形式（中の実行ファイル+データ）で出力できます

## あなただけが書くファイル（AI は提案まで）

次のファイルは**あなたが著者**です。AI はテンプレのコピー・質問リスト・転記文の提案までしかせず、**中身を書き込みません**（あなたの意思の一次記録を AI の文章と混ぜないため）。

Table 9



| ファイル                                               | 説明                                                                                |
|----------------------------------------------------|-----------------------------------------------------------------------------------|
| docs/conventions.md                                | プロジェクト規約。⑨で AI が質問リストを出すので、あなたが記入して確定                                             |
| docs/domain-knowledge.md                           | 業務知識。AI が「この内容で DK に追記しては」と転記文を提案するので、あなたが貼る                                      |
| docs/exception-policy.md                           | 例外の決定の登録簿。あなたが <code>hazards.py add-policy</code> コマンドを実行して登記（コマンド例は未決定キューに出ています） |
| docs/templates/（spec.md・test-spec.md）              | 仕様書の項目立てと書き方。プロジェクトに合わせて編集できます（機械が守る固定契約だけは変更不可）                                  |
| docs/prompts/（1-spec・2-testspec・3-testcode・4-impl） | 工程ごとの AI への指示。「このプロジェクトではここを重点的に」「この指摘はもう繰り返さないで」を書く場所（次項）                        |
| ISSUE の「回答（人が記入）」欄                                 | じっくり書きたい裁定はファイルで                                                                  |
| docs/review-feedback.md                            | 修正依頼。直接書いても <code>review_actions.py request-changes</code> でも同じ                   |

そのほかの編集可否:

Table 10

| ファイル                 | 手編集 | 説明                                                       |
|----------------------|-----|----------------------------------------------------------|
| docs/specs/（仕様書）     | △   | 編集可。ハッシュ連鎖が下流を stale にして再確認が走る（設計どおり）                    |
| WBS・テスト結果・完了検証レポート   | ✗   | 自動生成。次の再生成で消える                                           |
| 変数辞書・未決定キュー・矛盾候補のページ | ✗   | 自動生成。意味を直すときは <code>variables.py revise</code> か、チャットで頼む |

ファイルに直接書いた内容は、次にどのフェーズを起動したときに AI が自動で拾います（全 skill は起動時に「回答済み ISSUE・修正依頼・規約変更」をスキャンしてから本処理に入ります）。チャットで伝える場合、AI は転記文を提案するので、あなたが貼って確定します。

## AI への指示をプロジェクトごとに調整する（docs/prompts/）

同じ指摘を毎回している——「単位の変換は式まで書いて」「テストは 0 近傍を必ず入れて」——なら、それは指示を書く場所があるというサインです。工程ごとに 1 ファイル用意してあり、①～④は起動のたびにここを読んでから書き始めます（作り直し・改訂・バッチ実行でも同じ）。

Table 11

| ファイル                   | 効く工程  | 書くこと                           |
|------------------------|-------|--------------------------------|
| docs/prompts/1-spec.md | ① 仕様書 | 重点・用語の統一・繰り返さないでほしい指摘・手本にする仕様書 |

| ファイル                       | 効く工程     | 書くこと                                    |
|----------------------------|----------|-----------------------------------------|
| docs/prompts/2-testspec.md | ② テスト仕様  | 必ず入れるケース観点・期待値の決め方・ケース名の付け方             |
| docs/prompts/3-testcode.md | ③ テストコード | fixture やパラメータ化の書き方・使ってよいライブラリ・手本にするテスト |
| docs/prompts/4-impl.md     | ④ 実装     | 実装の構造や分割の方針・避けたい API・性能上の注意・手本にするモジュール  |

```
ledger init-templates    # 雛形を配置（初回。既にあるものは上書きしません）
ledger prompts          # どの工程に書いたか、まだ雛形のままかを一覧
```

- 雛形のまま（案内コメントだけ）なら「個別指示なし」として扱われます。必要な工程だけ書けば十分です
- 書けるのは「どう書くか」。工程の順序・読んでよい入力の制限・必須項目・Confidence と根拠の必須・承認の要否といった**土台の規則は変えられません**（AI はそこに反する指示には従わず、あなたに報告します）。土台を変えたいときは skill 自体の改版になるので、開発者に相談してください
- **項目立て（節の構成）は docs/templates/ 側、規約（型対応・丸め・命名・docstring）は docs/conventions.md 側**です。ここは「書き方の癖」を足す場所

使いこなしのコツ: レビューで 2 回同じことを指摘したら、その内容を「繰り返さないでほしい指摘」に 1 行足してください。3 回目からは指摘が要らなくなります。

## ⑦ 分析・改善の回し方

⑥が pass した後、`/legacy-7-analyze` で開始します。1 施策 = 1 枚の施策票 (Markdown ファイル) = 1 回の改善サイクル、という進め方です。

1. **計測・走査:** 機械が `profile / radon / ruff / bandit / pip-audit` を実行し、その実測値を見て AI が候補を `analysis.md` に列挙します。このとき**代表ワークロード (実データ規模の入力)**を用意しておく  
と計測が本命になります。単体テスト負荷だけではホットスポットを断定しません

**代表ワークロードの置き方:** プロジェクトルートに `bench.py` を置き、**引数なしで `python bench.py` を実行すると代表的な処理が一通り走るように**書いてください (`if __name__ == "__main__":` から呼ぶだけで構いません)。計測はこれを `cProfile` の下で実行する形で行います。数十秒〜数分程度で終わる規模が扱いやすいです。無ければテスト負荷でのスモーク計測になり、その旨がレポートに明記されます。

2. **項目確定:** 着手する候補が施策票 (`docs/improvements/OPT-001.md` など) に昇格。票には**目的・検証可能な成功基準 (baseline→目標)**・改善仕様が書かれる。**あなたはこの票を承認する**
3. **イタレーション:** AI が適用 (1 施策=1 コミット) → テスト全 pass 確認 → 再計測 → 票に実測と判定を記録。全基準達成なら **achieved** 、未達なら巻き戻して振り返り
4. WBS の「⑦改善イタレーション」表で全施策の達成状況 (//—) がトレースできる

Rust(PyO3) 化のような大きな施策も、AI が 4 段階基準 (CPU バウンドか → NumPy で足りないか → 純粋関数か → 保守コスト明記) で根拠付きの提案を出すので、票の承認で判断してください。

## get\_rate のフォールバック読込を分離し複雑度を下げる

### 目的（Plan）

`get_rate` はプロジェクト唯一の複雑度B評価（CC=8）。マスタ読込と検索という2つの責務が同居しているため、将来マスタ形式が変わったときの修正範囲を読込側に閉じ込められるよう、読込を `private` 関数 `_load_rate_master()` に分離して両関数をA評価にする。

### 成功基準（検証可能な形で定義する）

| # | 指標                              | baseline | 目標      | 実測(after)                                                       | 判定 |
|---|---------------------------------|----------|---------|-----------------------------------------------------------------|----|
| 1 | radon CC: <code>get_rate</code> | B(8)     | A(5)以下  | A(5)                                                            | ✓  |
| 2 | radon CC: 分離後の全関数               | —        | 全てA     | <code>get_rate</code> A(5), <code>_load_rate_master</code> A(5) | ✓  |
| 3 | radon MI: <code>rates.py</code> | A(83.47) | A維持     | A(80.02)                                                        | ✓  |
| 4 | ⑤テスト (15ケース)                    | 全pass    | 全pass維持 | 15 passed                                                       | ✓  |

### 改善仕様（Do の設計）

- `rates.py` に `_load_rate_master() -> int` を新設（0=成功/読込済み扱い、2=ファイル無し）。`RATES.DAT` のオープン・（A1,F5.3）パース・10件打ち切り・STATE への格納をここへ移す
- `get_rate` は「未ロードなら `_load_rate_master()`、エラー2なら（0,0,2）、あとは検索」に縮む
- 挙動保存の根拠: 外部から見た入出力（引数・戻り値・STATE の事後状態・ファイルアクセス）は一切変えない。関数境界の移動のみ。テスト15ケース（②F0001-TC-001～006を含む）が判定する
- 付き `private` のため Sphinx の公開APIには現れない（ドキュメント影響なし）

### 検証記録（Check）

- 2026-07-25 適用。 `_load_rate_master()` を新設し読込処理を移動、 `get_rate` は判定+検索のみに
- `pytest tests -q` → 15 passed（②由来の特性テスト全維持。挙動保存を確認）
- radon cc: `get_rate` B(8) → A(5)、 `_load_rate_master` A(5)。B評価消滅
- radon mi: A(83.47) → A(80.02)（Aランク維持）

### 振り返り（Act）

図 9: ⑦施策票の例（REF-001）。目的→成功基準（baseline→目標→実測→判定）→検証記録→振り返りが1枚に揃い、目的が達成されたかが後から機械的に追える

#### 目次

##### 目的（Plan）

成功基準（検証可能な形で定義する）

改善仕様（Do の設計）

検証記録（Check）

振り返り（Act）

Table 12

| 症状                                                                                                | 意味                                          | 対処                                                                                                                       |
|---------------------------------------------------------------------------------------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| WBS に  ISSUE-xxx | ④⑤ループが上限到達、裁定待ち                             | ISSUE に回答 → unblock → ⑤再トリガ                                                                                              |
| WBS に stale△                                                                                      | 上流 (①) が改訂され②が古い                            | /legacy-2-testspec で再生成 → 再承認                                                                                            |
| WBS に △改変                                                                                         | freeze 後にテストコードが変わった                        | 意図した変更なら③で再 freeze。心当たりがなければ調査                                                                                           |
| tests/ 編集が拒否された                                                                                   | hook が正常動作している                              | テスト側の疑義は ISSUE 経由で (正規経路)                                                                                                |
| 再レンダリングが失敗する                                                                                      | 配信サーバが _site をロック                           | serve_site.py で配信する (cwd を動かさないでロックしない)。素の http.server を使うなら cwd を _site の外にして --directory 指定                            |
| ブラウザが古い内容のまま                                                                                      | ブラウザのキャッシュ                                  | serve_site.py はキャッシュを無効にして配る。素の http.server の場合は強制リロード (Ctrl+F5)                                                         |
| 配布した EXE が古い進捗を表示する                                                                               | EXE の中身はビルド時点のスナップショット                      | build_viewer.py で作り直して渡し直す                                                                                               |
| 図がソースのまま表示される                                                                                     | quarto render docs を直接叩いた                   | render_site.py で出し直す (Mermaid は影コピー経由でのみ描画される)                                                                           |
| サイト更新に時間がかかる                                                                                      | 全体レンダになっている (初回・テーマ変更・変更ページ多数)              | 2 回目以降は差分レンダ (変わったページだけ) で数十秒。放置でよい                                                                                      |
| サイト内検索に新しいページが出ない                                                                                 | 差分レンダは検索索引を更新しない                            | render_site.py --root . --full を 1 回実行 (夜間バッチ後などの節目で)                                                                    |
| WBS の関数名が何行にも折れる                                                                                  | docs/wbs.css が無い / index.qmd を手編集した         | AI に「WBS を作り直して」と言う (python <LR>/scripts/ledger.py --root . wbs)。CSS はテンプレ (<LR>/assets/templates/wbs.css) から docs/ にコピー |
| PDF だけ図が出ない                                                                                       | Mermaid の画像化に Chromium 系ブラウザが要る             | python <qtpdf.py> doctor で確認。HTML 側はブラウザが描くので影響なし                                                                        |
| PDF 生成が「qtpdf.py が見つからない」で失敗                                                                      | 合本 PDF 用の quarto-typst-pdf を配置していない (HTML だ | quarto-typst-pdf/ を <LR> の隣 (.claude/skills/) にコピーす                                                                      |

| 症状                                                   | 意味                                                                      | 対処                                                                                                                                                                                                         |
|------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | けなら不要なので、既定では入れていません)                                                   | る。または <code>pdf_book.py --qtpdf &lt;パス&gt;</code> で直接指定                                                                                                                                                    |
| ⑩で「完全性突合 NG」                                         | 抽出器の 2 系統カウントが不一致（変則的なソース）                                              | <code>extract-report.json</code> の該当ファイルを AI に調査させる。判断がつかなければ ISSUE                                                                                                                                        |
| ①②の機械レビュー NG が報告された                                  | ハルシネーション・省略を機械が検知（正常動作）                                                 | AI が自分で直してから再提出してくる。NG 付きの成果物を承認しない                                                                                                                                                                        |
| 案内パネルに書いてあるコマンドを実行しても NG と言われる                       | 機械レビュー NG が残っている（承認はどの入口でも拒否される設計）                                      | パネル（または CLI の出力）の NG 理由を読み、チャットで AI に自己修正させてから承認する                                                                                                                                                         |
| バッチで、応答はあるのにファイルが更新されない（全件が同じ理由で NG）                 | headless がツールの実行許可を得られていない                                              | 応答ログに「許可」「permission」の文言が出る。 <code>settings.json</code> の <code>permissions.allow</code> を拡充するか、閉じた環境なら <code>--skip-permissions</code> を付ける                                                               |
| バッチが「データ異常のため停止」と出た                                  | <code>data/functions.json</code> 等が JSON として壊れている（AI の編集ミス）             | 停止理由に壊れた位置（行・列）が出る。 <code>git diff data/functions.json</code> で直前の変更を確認して修復、または <code>git checkout -- data/functions.json</code> で戻す。hook ( <code>guard_json.py</code> ) を登録していれば、そもそもこの状態になる前に AI へ差し戻される |
| 連続実行が「連続 3 件失敗」で止まった                                 | 環境異常の疑いで安全停止した（1 件ずつの失敗とは別扱い）                                           | 結果パネルの NG 行から応答ログを読む。API キー・レート・claude の解決などを確認してから再開                                                                                                                                                      |
| 残タスクに、さっき失敗した関数がまだ「次 N」で出ている                         | スキップはそのバッチ限りの扱いで、状態としては未着手のまま                                           | 意図どおり。いま拾い直すなら ★優先、次回バッチなら自動で対象に戻る                                                                                                                                                                         |
| PowerShell では claude が動くのに pipeline.py から動かない/全件失敗する | claude が PowerShell のプロファイルでしか解決できない・実体が .ps1・Python を起動したシェルの PATH が違う | <code>pipeline.py</code> は起動前に claude を実測チェックして原因を表示する。PowerShell で <code>(Get-Command claude).Source</code> を調べ、 <code>--claude-cmd "&lt;実体パス&gt;"</code> で明示するのが確実                                       |
| WBS のファイル別ページが出ない                                    | 旧テンプレの <code>_quarto.yml</code> に <code>wbs/*.qmd</code> が無い            | <code>render_site.py</code> が自動追記するのでそのまま再実行。恒久                                                                                                                                                            |

| 症状 | 意味 | 対処                             |
|----|----|--------------------------------|
|    |    | 対応はテンプレから<br>_quarto.yml を差し替え |



## PDF 出力

種別ごとの合本 PDF は次で生成します（フォールバック含め自動処理）。

```
python <LR>/scripts/pdf_book.py specs      --root . --output pdf/関数仕様書.pdf      --title  
関数仕様書  
python <LR>/scripts/pdf_book.py test-specs  --root . --output pdf/テスト仕様書.pdf  --title  
テスト仕様書  
python <LR>/scripts/pdf_book.py test-results --root . --output pdf/テスト結果報告書.pdf --  
title テスト結果報告書
```

生成後は `python <qtpdf.py> check <pdf>` で豆腐・はみ出し（文字化け・紙外へのはみ出し）の機械チェックができます。`<qtpdf.py>` は `quarto-typst-pdf/scripts/qtpdf.py`（skills リポジトリ、またはコピー先の `.claude/skills/`）です。

## 付録: 成果物とデータの全体像

|                           |                                                |
|---------------------------|------------------------------------------------|
| docs/                     |                                                |
| index.qmd                 | WBS（自動生成・手編集禁止）                                |
| wbs/                      | 200 関数超で自動生成されるファイル別明細ページ（手編集禁止）               |
| conventions.md            | プロジェクト規約（⑩で確定・人が編集可）                           |
| domain-knowledge.md       | 裁定の蓄積（人が編集可）                                   |
| variables.qmd             | 変数辞書ページ（自動生成・手編集禁止。承認はチャット / CLI）              |
| dict-conflicts.md         | 辞書と承認済み仕様書の矛盾候補（自動生成）                          |
| exception-policy.md       | 例外ポリシー登録簿 EP-xxx（あなたが承認する規約）                   |
| exception-queue.md        | まだ決めていない例外の質問キュー（自動生成）                         |
| specs/F-xxxx.md           | ① 関数仕様書（skeleton→draft→reviewed）               |
| test-specs/F-xxxx.md      | ② テスト仕様書（generated→approved）                   |
| test-results/F-xxxx_日時.md | ⑤ 結果報告書（毎回新規・自動生成）                             |
| issues/ISSUE-xxx.md       | 質問と裁定（回答欄はあなたが記入可）                             |
| improvements/XXX-nnn.md   | ⑦ 施策票（proposed→approved→applied→verified）      |
| completion-check.md       | ⑥ 完了検証（自動生成）                                   |
| data/                     |                                                |
| functions.json            | ⑩の解析結果（正データ。Fortran は機械抽出が生成・マージ）              |
| extract-report.json       | ⑩機械抽出の監査ログ（完全性突合・推定呼出・未解決名）                    |
| ledger.json               | ハッシュ・ブロック状態（スクリプト専用）                           |
| variables.json            | 変数辞書（スクリプト専用。承認内容もここ）                          |
| hazard-map.json           | 0 割等 × 適用ポリシーの突合結果（スクリプト専用）                    |
| src/ , tests/             | ④実装 / ③テスト（④⑤中は hook が tests/ を保護）             |
| .legacy-reverse/          | 実行時の作業データ（git 管理外・消しても成果物は失われない）               |
| pipeline-status.json      | バッチ実行状況ページが表示するライブ状態                           |
| pipeline-log.jsonl        | 1 行 1 試行の実行ログ（結果・所要・コスト）                       |
| agent-logs/F-xxxx.txt     | エージェント応答の全文（失敗原因の一次情報）                         |
| batch-priority.json       | ★優先の割り込み順（バッチ状況ページの★／pipeline.py priority が書く） |
| run.lock                  | 実行枠のロック（バッチとブラウザ実行の二重起動を防ぐ）                    |

付録: 画面に出る記号の一覧

Table 13

| 記号  | 意味                             | あなたの対応                    |
|-----|--------------------------------|---------------------------|
| ✓   | 完了                             | なし                        |
| ▲   | 作業中（draft・generated など、承認前の状態） | 承認する                      |
| □   | 未着手                            | 着手する（または連続実行に任せる）         |
| ➡   | 裁定待ちで止まっている                    | ISSUE に回答して裁定する（最優先）      |
| ⚠   | 要再確認（下に 4 種類）                  | 種類による                     |
| ✖   | 機械レビュー NG／⑦の基準未達               | AI の自己修正を待つ（✖のまま承認はできません） |
| ●●● | 仕様項目の Confidence（確認済／推測／仮定）    | ●●● は中身を見る                |

⚠ は文脈で 4 つの意味があります（画面の該当欄に理由が出ます）：

Table 14

| 出る場所            | 意味                   | 対応                             |
|-----------------|----------------------|--------------------------------|
| WBS の②列（stale⚠） | ①が改訂されて②が古くなった       | ②を再生成して再承認                     |
| WBS の③列（⚠改変）    | freeze 後にテストコードが変わった | 意図した変更なら③で再 freeze。心当たりがなければ調査 |
| WBS の①列（辞書⚠）    | ①を書いたあとに変数の意味が改訂された  | 「辞書と仕様書の矛盾候補」を見て、必要なら①を改訂      |
| ②テスト仕様書の中（⚠未確定） | 期待値が決められないケースが残っている  | 質問に答える（残ったままでは承認できません）         |

Table 15

| 用語                     | 意味                                                                                   |
|------------------------|--------------------------------------------------------------------------------------|
| <b>WBS</b>             | 進捗のホーム画面 ( <a href="#">docs/index.qmd</a> → サイトのトップ)。全関数の状態が一覧できます                   |
| <b>台帳 / ledger</b>     | 同じものです。ハッシュや裁定待ち状態を持つスクリプト専用のファイル ( <a href="#">data/ledger.json</a> )。人は触りません       |
| <b>freeze</b>          | ③完了時にテストコードのハッシュを台帳に記録すること。以後の改変が機械検知できるようになります                                      |
| <b>stale</b>           | 上流が改訂されて下流が古くなった状態。自動で検知されて ⚠ が出ます                                                   |
| <b>headless Claude</b> | 画面を持たない <code>claude</code> コマンドのこと。無人バッチはこれを裏で 1 関数ごとに 1 回ずつ起動しています                 |
| <b>案内パネル</b>           | 仕様書などの HTML ページの本文先頭に焼き込まれる表示 (機械レビュー結果と返答方法)。表示だけで、操作ボタンはありません                      |
| <b>一斉レビュー表</b>         | draft の仕様書をまとめて処理するための一覧ページ ( <a href="#">docs/spec-review.md</a> )                  |
| <b>機械レビュー</b>          | 人に見せる前にスクリプトが行う検査 (引用の实在・記入漏れ・トレーサビリティ)                                              |
| <b>変数辞書</b>            | 「1 変数=1 つの意味」を確定させた表 ( <a href="#">docs/variables.qmd</a> )。承認した意味は仕様書の入出力表へ自動転記されます |
| <b>rank (A/B/C/D)</b>  | 変数の意味の確からしさ。AI の申告ではなく、引用された根拠の種類から機械が決めます                                           |
| <b>hazard</b>          | 0 割・平方根の負値・対数の 0/負値・変数添字など、Python では止まってしまう箇所。⑥が機械で洗い出します                            |
| <b>EP-xxx</b>          | hazard の扱いの決定 (例外ポリシー)。あなたが承認して積み上げる規約で、同種の全箇所に効きます                                  |
| <b>フロー</b>             | 作業スコープ。main が複数あるときなどに「このフローだけ進める」と指定できます                                            |